

*Same as well
Just better*

SAME YOU SINCE 2009
JUST BETTER

Before we start

Let's rescue our Nginx
Server

Lesson 3

Git & Version Control



Remember him?)

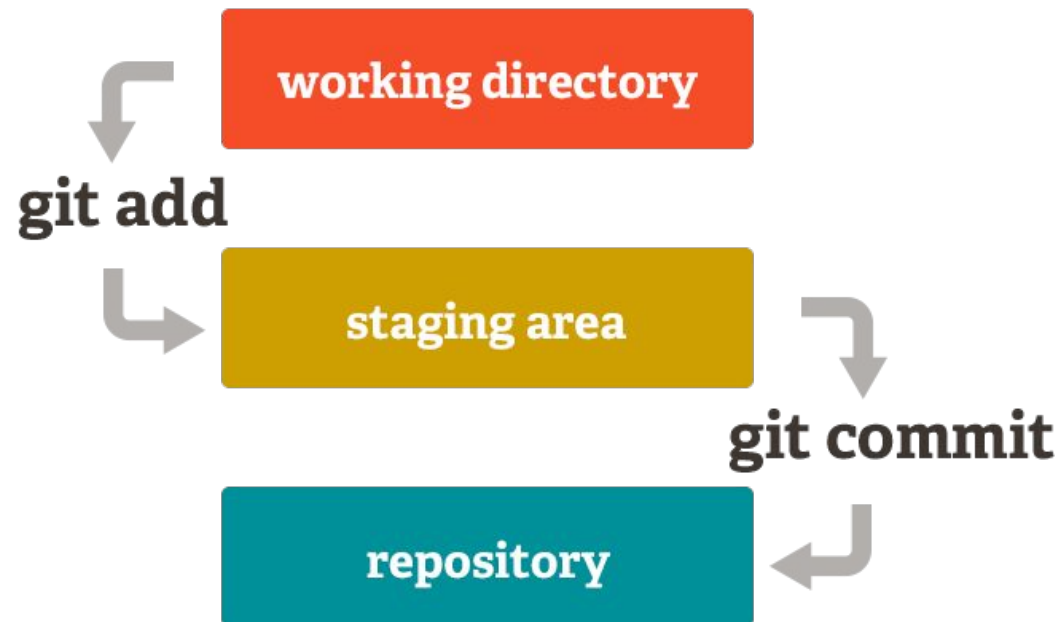
Linus Torvalds: Creator of Linux & Git.

Built Git in 2005 to manage the massive Linux Kernel project because existing tools were too slow.

- **The Problem:** Naming files
script_final_v2_FINAL_really.sh.
- **The Solution:** Git. Distributed, fast, and tracks every single change.



- **Working Directory:** Your actual files on your computer. It's a sandbox.
- **Staging Area:** A rough draft space. "I want these specific changes in my next snapshot."
- **Local Repository (.git):** The permanent database of your committed snapshots.



git init: Creates a hidden .git folder. Your directory is now a Git repo.

git status: Your best friend. Tells you what branch you are on and what files are modified, staged, or untracked.



```
# The First 10 GIT COMMANDS I Type in Every New Project
```

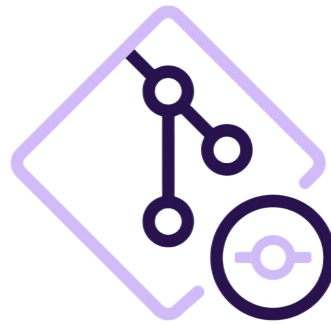
```
git init
git status
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/username/repository.git
git push -u origin main
git log
```



git add <file>: Moves a file to the Staging Area. Use `git add .` to stage everything.

git commit -m "message": Takes a snapshot of the staging area.

git log: Shows the history of your commits, who made them, and when.



Git Commands

git init



Initializes a new Git repository in the current directory.

```
$ mkdir railsapp
$ cd railsapp
$ git init
```

git add



Stages changes in the current directory and sub directories.

```
$ echo "# RAILS APP" > README.md
$ echo "Demo" >> README.md
$ git add README.md
```

git commit



Commits staged changes with a commit message.

```
$ git add app.css
$ git commit -m "Initial commit"
```

git push



Pushes local changes to a remote repository.

```
$ git add .
$ git commit -m "Update files"
$ git push origin main
```

git pull



Pulls changes from a remote repository and merges into the local repository.

```
$ git status
$ git pull origin main
$ git log --oneline
```

git remote



Add a remote repository, view it, and rename it.

```
$ git remote add origin <url>
$ git remote -v
$ git remote rename origin upstr
```

git branch



List all branches, create a new branch with a specified name, and confirm it's created.

```
$ git branch
$ git branch feature-login
$ git branch
```

git fetch



Retrieves the latest data from a remote repository - but it doesn't integrate any of this new data into your working files.

```
$ git fetch origin
```

git checkout



Switches to the specified branch.

```
$ git branch
$ git checkout feature-login
$ git branch
```

git merge



Merges the specified branch into the current branch (in this case main).

```
$ git checkout main
$ git merge feature-login
$ git log --oneline
```

git status



Displays the status of the repository, including any uncommitted changes

```
$ git status
$ echo "Data Added" >> hello.txt
$ git status
```

git reset



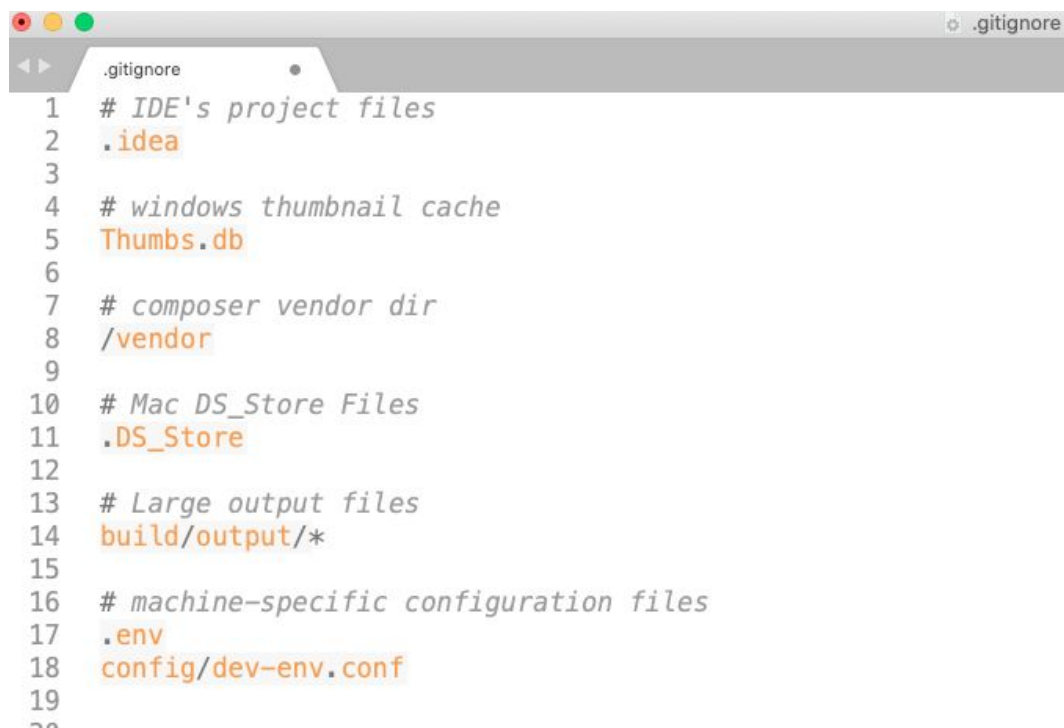
Resets the current branch to the specified commit

```
$ git log --oneline
$ git reset --hard <commit-hash>
$ git status
```


The Problem: We don't want to track logs, compiled binaries, or secret keys.

The Solution: The .gitignore file.

How it works: Any file or folder pattern listed inside .gitignore becomes invisible to Git.



```
.gitignore
1  # IDE's project files
2  .idea
3
4  # windows thumbnail cache
5  Thumbs.db
6
7  # composer vendor dir
8  /vendor
9
10 # Mac DS_Store Files
11 .DS_Store
12
13 # Large output files
14 build/output/*
15
16 # machine-specific configuration files
17 .env
18 config/dev-env.conf
19
20
```

Remotes: Versions of your project hosted on the internet (GitHub, GitLab, Bitbucket).

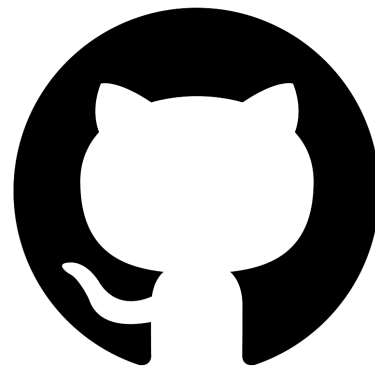
git remote add origin <URL>: Links your local repo to a remote server.

git push: Uploads your local commits to the remote repository.

git pull: Downloads changes from the remote repository to your local machine.



GitLab



GitHub



Bitbucket

Now, let's do some real work!



Now, it's time for the Quiz



- Next Lesson: Collaborative Development (Branching, Merge Conflicts, Pull Requests).
- Prep Work: Ensure your GitHub account is fully set up and you remember your credentials! We will be simulating a team environment.

Questions?

CONTACT US